

The World Islamic Sciences and Education University

جامعة العلوم الإسلامية العالمية

Faculty of Information Technology

كلية تكنولوجيا المعلومات



GRADUATION PROJECT

Title

Smart Project Management

Students

Andalus Mohammad Mahmoud Alqaisi: 3230605033

Fatima Salah Adeen Jber: 3220605040

Shahed Omar Matrok Alsnaid: 3220605045

Supervisor

Adeeb Saaidah

SEMESTER II

2025/2026

Acknowledgements

We would like to express our sincere gratitude to everyone who supported us throughout the development of this graduation project. First and foremost, we would like to thank our supervisor for his continuous guidance, valuable feedback, and encouragement during all stages of this project. His support played a significant role in shaping our ideas and improving the quality of our work.

We are also grateful to the faculty members of the Information Technology department for providing us with the knowledge and skills required to complete this project successfully.

Special thanks go to our families and friends for their patience, understanding, and moral support throughout this journey. Their encouragement motivated us to overcome challenges and complete this project with dedication.

Finally, we would like to thank all those who contributed directly or indirectly to the completion of this project.

Abstract

In modern organizations, effective project management is essential for ensuring productivity, coordination, and successful task completion. However, many companies still rely on manual processes or multiple unintegrated tools to manage tasks, which often leads to inefficiency, lack of transparency, and difficulty in tracking performance.

This project presents a Smart Project Management System, a web-based platform designed to improve collaboration between team leaders and team members by providing a centralized environment for managing projects and tasks.

The system allows team leaders to create projects, assign tasks to employees, define deadlines, and set task complexity levels. At the same time, team members can access their assigned tasks, update their progress, and track the time spent on each task. The system also includes real-time tracking features that enable leaders to monitor task completion percentage, analyze time consumption, and evaluate overall team performance.

In addition, the system provides an interactive dashboard that visualizes project progress through charts and reports, helping leaders make informed decisions based on accurate data. The database of the system is designed using PostgreSQL to ensure efficient data storage, integrity, and scalability for handling multiple users, projects, and tasks.

The proposed system integrates task management, time tracking, and performance monitoring into a single platform, which reduces complexity and improves workflow organization. As a result, the system enhances transparency, increases productivity, and provides a practical solution for managing projects in real-world environments.

Table of Contents

Acknowledgements	I
Abstract	II
Table of Contents	III
List of Tables.....	V
List of Figures	V
List of Abbreviations.....	VI
CHAPTER 1 INTRODUCTION	1
1.1 Overview	1
1.2 Problem Statement	2
1.3 Project Aim	2
1.4 Project Objectives	2
1.5 Significance of the Project	2
1.6 Scope of the Project	3
1.7 Expected Outcomes.....	4
1.8 Chapter Summary.....	4
CHAPTER 2 LITERATURE REVIEW	5
2.1 Overview	5
2.2 Related Work	5
2.2.1 Discussion of What Has Been Done Along the Line of the Problem	5
2.2.2 Issues and Challenges Faced by Other Projects.....	6
2.2.3 Critical Evaluation (Gap Analysis).....	6
2.3 Summary	7
CHAPTER 3 METHODOLOGY	8
3.1 Overview	8
3.2 Feasibility Study.....	8
3.2.1 Technical Feasibility.....	8
3.2.2 Economic Feasibility	8
3.2.3 Operational Feasibility.....	9
3.3 Risk Analysis	9
3.4 Methodology Process	10
3.4.1 Requirements	10
3.4.1.1 Functional Requirements.....	10
3.4.1.2 Non-Functional Requirements.....	11
3.4.2 System Design	12

3.4.3	Implementation	13
3.4.4	Testing	14
3.4.5	Deployment	14
3.5	Chapter Summary.....	15
CHAPTER 4	DESIGN MODELS	16
4.1	Overview	16
4.2	Context Diagram (Level-0)	16
4.3	Data Flow Diagram (Level-1)	17
4.4	Use Case Diagram.....	18
4.5	Activity Diagram.....	19
4.6	Entity-Relationship (ER) Diagram.....	21
4.7	Relational Model	22
CHAPTER 5	EXPERIMENTS AND RESULTS.....	24
5.1	Overview	24
5.2	Testing Methodologies.....	24
5.2.1	Unit Testing Results	25
5.2.2	Integration Testing Results	28
5.2.3	System Testing Results.....	30
5.2.4	Acceptance System Results	32
5.3	Discussion and Evaluation	33
CHAPTER 6	CONCLUSION AND FUTURE WORKS	35
6.1	Overview	35
6.2	Summary about the Project.....	35
6.3	Achieved Objectives.....	36
6.4	Main Contributions of the Work.....	38
6.5	Limitations.....	39
6.6	Future Work.....	40
6.7	Chapter Summary	43
References		43
Appendix A	Source Code.....	44
A.1	Database Connection (db.js)	44
A.2	Server Configuration (app.js)	44
A.3	Authentication Middleware(auth.js)	45
A.4	Authentication Controller(authController.js).....	46
A.5	Frontend API Service(api.js)	47
A.6	Frontend Routing(app.js)	48

Appendix B-User Satisfaction Survey Results	48
B.1 Participant Profile	48
B.2 Quantitative Ratings.....	49
B.3 System Adoption Intent	49
B.4 Deployment Readiness Assessment.....	49

List of Tables

Table 2.1	Comparison of Challenges in Existing Project Management Tools.....	6
Table 3.1	Risk Analysis	9
Table 4.1	Entity Descriptions — Smart Project Management System.....	21
Table 5.1	Unit Testing – Authentication Module (authController.js)	25
Table 5.2	Unit Testing – Project Module (projectController.js)	26
Table 5.3	Unit Testing – Task Module (taskController.js)	27
Table 5.4	Unit Testing – Notification & Report Modules.....	28
Table 5.5	Integration Testing Results.....	29
Table 5.6	System Testing – Authentication & User Management	30
Table 5.7	System Testing – Project & Task Management Workflows	30
Table 5.8	System Testing – Notifications & Reports.....	31
Table 5.9	Acceptance Testing Results	32
Table 5.10	Testing Results Summary.....	34
Table 6.1	Achieved Objectives	36
Table 6.2	System Limitations.....	39
Table 6.3	Proposed Future Enhancements	40
Table B.1:	Participant Roles	48
Table B.2:	Quantitative Survey Results	49
Table B.3:	Would You Use This System?	49
Table B.4:	Deployment Readiness	49

List of Figures

Figure 4.1	Context Diagram (Level-0) — Smart Project Management System	16
Figure 4.2	Data Flow Diagram (Level-1) — Smart Project Management System	17
Figure 4.3	Use Case Diagram — Smart Project Management System.....	18
Figure 4.4	Activity Diagram — Smart Project Management System	20
Figure 4.5	Entity-Relationship (ER) Diagram — Smart Project Management System.....	21
Figure 4.6	Relational Model — Smart Project Management System	23

List of Abbreviations

Abbreviation	Definition
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DB	Database
DFD	Data Flow Diagram
ER	Entity-Relationship
FK	Foreign Key
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IT	Information Technology
JS	JavaScript
JSON	JavaScript Object Notation
JWT	JSON Web Token
MVC	Model-View-Controller
PK	Primary Key
REST	Representational State Transfer
SaaS	Software as a Service
SQL	Structured Query Language
UI	User Interface
URL	Uniform Resource Locator
UX	User Experience
WISE	World Islamic Sciences and Education
WISE U	World Islamic Sciences and Education University

CHAPTER 1

INTRODUCTION

1.1 Overview

Project management has become an essential part of how companies operate today, especially when teams need to work together to reach their goals on time. As projects grow in size and complexity, relying on manual methods or scattered tools is no longer enough to keep everything organized. Many teams still use emails, spreadsheets, or separate applications to manage their work, which leads to confusion, missed deadlines, and difficulty in knowing the real status of any project.

From our experience working on this project, we noticed that both team leaders and members face real challenges when there is no proper system in place. Leaders need a clear way to create projects, assign tasks, set deadlines, and monitor how their team is doing. Members need to know exactly what is expected from them, update their progress, and record the time they spend on each task. These observations motivated us to build the Smart Project Management System, a web-based platform that brings all of this together in one place.

The system allows the team leader to create and manage projects, organize team members, assign tasks with deadlines and complexity levels, and track overall progress. At the same time, members can log in, view their assigned tasks, update completion percentages, and record their working time. The system also includes monitoring features that give the leader a clear picture of each member's performance.

One important feature is the dashboard, which presents project progress through visual charts and reports. Instead of going through raw data, the leader can quickly see task distribution, completion rates, and where delays are occurring. This makes decision-making faster and more reliable.

The use case diagram in Figure 4.2 shows the responsibilities of each role, where the leader manages projects and tasks while members update their work and track time. The activity diagram in Figure 4.3 describes the full workflow from login all the way to task completion and report generation.

Overall, this system provides a practical solution for small and medium-sized teams that need a reliable way to manage projects and evaluate performance. By combining task management, time tracking, progress monitoring, and reporting into a single platform, the system reduces the confusion that comes from using separate tools and creates a more organized and transparent work environment.

1.2 Problem Statement

In many companies and team-based environments, managing projects is still done manually or through separate, unconnected tools. This makes it difficult for team leaders to know the real status of tasks, track how much time employees are spending, and evaluate their performance in a reliable way. Without a centralized system, deadlines get missed, responsibilities become unclear, and communication between leaders and members breaks down.

The main issue is that there is no single place where leaders can create projects, assign tasks, and monitor progress, while members can view their work, update their status, and record their time. This gap is exactly what motivated us to build the Smart Project Management System, a platform that brings all of these needs together in one organized and easy-to-use environment.

1.3 Project Aim

The main aim of this project is to design and build a web-based Smart Project Management System that helps companies organize their projects and tasks in one place. Through this system, team leaders can assign work, monitor progress, and evaluate performance, while team members can view their tasks, update progress, and track working time. The system is meant to make collaboration easier and improve transparency within any team environment.

1.4 Project Objectives

The objectives of this project are derived from the system description and its intended outcomes. The main objectives are:

1. To design and implement a centralized web-based platform that allows team leaders to create and manage projects, assign tasks to members, and define deadlines and complexity levels.
2. To enable team members to access their assigned tasks, update their progress, and record the time they spend on each task in an organized and easy-to-use environment.
3. To support real-time performance monitoring by tracking task completion percentage, time spent, and overall team productivity for better decision-making.
4. To provide an interactive dashboard and reporting mechanism that visually presents project progress, task distribution, and team performance using charts and summaries.
5. To develop a scalable and practical system that improves collaboration, transparency, and accountability within small to medium-sized teams in real-world company environments.

1.5 Significance of the Project

The importance of this project lies in its ability to address real problems that teams and companies face during project execution. Success in any work environment does not only depend on assigning tasks, but also on monitoring progress, meeting deadlines, and understanding how

well each team member is performing. A system that combines all of these in one place provides real value for both leaders and members.

For team leaders, this project gives them better control over their projects. Instead of using manual methods or separate tools, the leader can create projects, assign tasks, set complexity levels, monitor progress, and evaluate performance using actual data. The use case diagram in Figure 4.2 shows these responsibilities clearly as the core functions of the leader role.

For team members, the system organizes their daily work and removes confusion about what needs to be done. Each member can log in, see their assigned tasks, update their progress, and track the time they spend. The activity diagram in Figure 4.3 describes this workflow step by step, from viewing tasks all the way to marking them as completed.

From a technical perspective, the project brings together several important concepts in one application, including user management, role-based access control, task tracking, time recording, and data visualization. The ER diagram in Figure 4.5 reflects this by showing all the core entities and how they connect, which confirms that this is a complete management system and not just a simple task list.

Overall, the significance of this project goes beyond building a software product. It shows how a well-designed web application can solve real organizational problems and support better decision-making through data and performance tracking.

1.6 Scope of the Project

This project covers the design and implementation of a web-based Smart Project Management System for small to medium-sized teams working in a company environment. The system runs through modern web browsers and focuses on role management, task assignment and tracking, time and performance monitoring, and dashboard-based visualization.

Within this scope, the system includes the following functional boundaries:

- User registration and login with role-based access control for leaders and members.
- Project creation, management, and team organization by the team leader.
- Task creation with title, description, deadline, status, and complexity level, along with assignment to specific members and monitoring of their completion progress.
- Progress tracking based on completion percentage and task status values such as Not Started, In Progress, and Completed.
- Time tracking by recording work duration for use in performance reports.

- Dashboard and report generation to visualize project progress and team performance using charts and summaries.

The scope does not include external enterprise integrations, mobile application deployment, advanced AI features, or large-scale ERP modules. The system is intentionally focused on providing a practical and manageable solution that addresses a real organizational need within the graduation project context.

1.7 Expected Outcomes

Upon completion, this project is expected to deliver a working web-based system that helps companies organize their projects and monitor employee performance more effectively. The main expected outcomes are:

- A centralized platform for creating and managing projects and tasks.
- Better visibility of task progress and completion status for both leaders and members.
- Accurate time tracking that supports performance evaluation and reporting.
- An interactive dashboard that helps leaders monitor productivity and compare team members.
- Improved communication of responsibilities and reduced confusion in daily workflows, as supported by the role-based flow shown in Figure 4.3.
- A complete database-driven system built with PostgreSQL and a Node.js backend, ready for real-world use.

1.8 Summary

This chapter introduced the Smart Project Management System, a web-based platform built to improve how teams manage their projects, track tasks, monitor time, and evaluate performance. The chapter covered the main problems that motivated this project, defined its aim and objectives, explained its significance for both leaders and members, and described the scope and expected outcomes. Overall, the system is designed to provide a centralized and efficient environment where projects can be handled more effectively and where leaders have access to clear data and visual reports to support their decisions.

CHAPTER 2

LITERATURE REVIEW

2.1 Overview

This chapter presents a review of existing literature and related systems in the field of project management, task tracking, and team collaboration tools. The purpose of this chapter is to examine what has already been developed in this area, identify the limitations and challenges of existing solutions, and highlight the gap that the proposed Smart Project Management System aims to fill. By analyzing prior work, this chapter provides a solid foundation for understanding the design decisions and features proposed in this project.

2.2 Related Work

Several existing systems and research studies have been developed in the domain of project management and task tracking. The following subsections discuss the most relevant ones.

2.2.1 Discussion of What Has Been Done Along the Line of the Problem

Trello

Trello is a popular web-based project management tool that uses a Kanban-style board system to organize tasks into lists and cards. It allows users to create boards for different projects, assign tasks to team members, set due dates, and attach files. Trello's visual interface makes it easy to understand the status of tasks at a glance. However, Trello lacks built-in time tracking features and does not offer a detailed performance monitoring mechanism for evaluating individual team members. It also has limited reporting capabilities and does not support role-based dashboards that separate the view for leaders and members [1].

Asana

Asana is another widely used project management platform that supports task creation, assignment, deadline setting, and progress tracking. It offers various project views including lists, boards, timelines, and calendars. Asana also includes a workload view that helps managers see how tasks are distributed among team members. Despite its many features, Asana can be complex for new users, and its free tier has significant limitations. Furthermore, Asana does not

natively provide a time-tracking feature, requiring users to integrate third-party tools for this purpose [2].

Jira

Jira, developed by Atlassian, is a powerful project management and issue-tracking tool primarily designed for software development teams. It supports agile methodologies such as Scrum and Kanban, and provides detailed reports including burndown charts and velocity reports. Jira allows team leaders to create sprints, assign story points to tasks, and monitor team velocity over time. However, Jira is known for its steep learning curve and complex configuration, making it less suitable for small teams or academic projects that do not require such advanced functionality [3].

Monday.com

Monday.com is a flexible work operating system that allows teams to build custom workflows for project management. It supports task tracking, progress visualization, and team collaboration with a highly customizable interface. Users can build dashboards using widgets to display key metrics such as task completion rates and workload distribution. While Monday.com is visually appealing and easy to use, it is primarily a commercial product with a high subscription cost, which makes it inaccessible for many academic or low-budget teams. Additionally, it does not emphasize time tracking as a core built-in feature [4].

2.2.2 Issues and Challenges Faced by Other Projects

Table 2.1: Comparison of Challenges in Existing Project Management Tools

Challenge	Trello	Asana	Jira	Monday.com
Integrated Time Tracking	X	X	X	X
Ease of Use / Low Learning Curve	✓	✓	X	✓
Affordable / Free Tier	✓	X	X	X
Individual Performance Monitoring	X	X	X	X
Role-Based Separation (Leader / Member)	X	X	X	X

2.2.3 Critical Evaluation (Gap Analysis)

The reviewed systems each address some aspects of project management but none fully meets the needs of small, role-based teams requiring integrated task management, time tracking, and

performance monitoring in one simple platform. The proposed Smart Project Management System fills this gap by combining all these features in a single web-based application that is easy to use and does not require any subscription.

2.3 Summary

This chapter provided an overview of existing project management tools and related research studies. It examined tools such as Trello, Asana, Jira, and Monday.com, and highlighted their key features as well as their limitations. The analysis revealed that existing solutions suffer from issues such as lack of integrated time tracking, high complexity, high cost, and insufficient performance monitoring.

These findings justify the development of the Smart Project Management System, which addresses the identified gaps by providing an integrated, role-based, web-based platform that combines task management, real-time progress tracking, time tracking, and performance analysis in a single, user-friendly system. The next chapter will describe the methodology used to develop this system.

CHAPTER 3

METHODOLOGY

3.1 Overview

This chapter describes the development approach, feasibility analysis, and the specific requirements that guided the design and implementation of the Smart Project Management System. It begins with a feasibility study evaluating the project from technical, economic, and operational perspectives, followed by a risk analysis identifying potential challenges and their mitigation strategies. The chapter then presents the Waterfall Software Development Life Cycle (SDLC) adopted for this project, detailing each phase: requirements gathering, system design, implementation, testing, and deployment.

3.2 Feasibility Study

A feasibility study was conducted to assess the viability of the proposed system from three perspectives: technical, economic, and operational.

3.2.1 Technical Feasibility

All technologies used in this project — React.js, Node.js, Express.js, PostgreSQL, and associated npm packages — are open-source, well-documented, and widely adopted. The development team has prior experience with JavaScript-based full-stack development. Hosting the application requires a standard web server and a PostgreSQL instance, both of which are readily available through cloud providers or local deployment. The use of JWT for stateless authentication and multer for file handling are mature, production-proven libraries, making the project technically feasible.

3.2.2 Economic Feasibility

The entire technology stack is free and open-source, eliminating all licensing costs. Development labor is the primary investment. For academic and small-team deployment, free-tier cloud

services such as Railway, Render, and Neon can host both the backend and the database, keeping operational costs at zero. Visual Studio Code is used as the development environment at no cost. Since no paid tools or services are required, the project is economically feasible.

3.2.3 Operational Feasibility

The system's two-role model (Leader and Member) maps directly to how most small teams are structured. The React.js interface is organized into distinct dashboards per role, minimizing the learning curve. Users can register, log in, manage projects, assign tasks, submit work, and track progress without requiring any special training. Swagger UI provides interactive API documentation at the /api-docs endpoint, supporting developer onboarding. These factors collectively confirm operational feasibility.

3.3 Risk Analysis

Several potential risks were identified during the planning phase. Table 3.1 summarizes the main risks along with their likelihood, impact level, and mitigation strategies.

Table 3.1: Risk Analysis

Risk	Likelihood	Impact	Mitigation Strategy
JWT token theft	Medium	High	Short token expiry (7 days) and HTTPS enforcement in production.
File upload abuse	Medium	High	File size limit (15 MB) and allowed file type validation enforced by multer middleware on the server side.
Database corruption	Low	High	Use of PostgreSQL transactions (BEGIN / COMMIT / ROLLBACK) for all multi-step operations such as task deletion cascading.
CORS misconfiguration	Low	Medium	Whitelist only the known frontend origin in the Express CORS configuration; block all other origins.
Concurrent write conflicts	Medium	Medium	PostgreSQL ACID guarantees prevent data inconsistency; last-write-wins for non-critical fields.
Scope creep	High	Medium	Requirements clearly defined and signed off before implementation began; changes require supervisor approval.
Time constraints	High	Medium	Project divided into clearly assigned tasks per team member following the Waterfall timeline.

3.4 Methodology — Waterfall SDLC

The Waterfall methodology was selected as the Software Development Life Cycle (SDLC) approach for this project. The Waterfall model is a linear and sequential approach in which each phase must be completed before the next one begins. This model was chosen for three reasons specific to this project. First, the system requirements were fully defined before development began through analysis of comparable tools (Trello, Asana, Jira, Monday.com) and structured discussion with the project supervisor, leaving little room for requirement changes mid-development. Second, the project has a fixed academic deadline, making the predictable, phase-gated structure of Waterfall more appropriate than iterative models that assume flexibility in scope and schedule. Third, the team of three members benefits from the clear phase boundaries that Waterfall enforces, since each member could be assigned distinct deliverables per phase without the overhead of sprint planning or backlog management. The five sequential phases applied are: Requirements Analysis, System Design, Implementation, Testing, and Deployment.

3.4.1 Requirements

Requirements were gathered through direct analysis of comparable tools (Trello, Asana, Jira, Monday.com), structured discussion among the development team, and consultation with the project supervisor. The following types of requirements were identified:

3.4.1.1 Functional Requirements

Functional requirements define what the system must do. The following functional requirements were identified and implemented:

1. **User Registration and Login:** Users can register with a full name, email, password, and role (leader or member). Passwords are hashed using bcrypt (salt factor 10) before storage. Successful login returns a JWT (7-day expiry) used for all subsequent authenticated requests.
2. **Project Management:** Leaders can create, update, and delete projects with a name, description, start date, end date, and status. Each project is associated with a single leader.

3. **Project Membership:** Members are linked to projects via the `project_members` join table. Leaders retrieve only their own projects; members retrieve only the projects they belong to.
4. **Task Management:** Leaders can create tasks specifying title, description, project, assignee, complexity level (1–5), deadline, and priority (Low / Medium / High / Urgent). Tasks can be updated, reassigned, or deleted.
5. **Task Progress Tracking:** Members update their task progress percentage (0–100). The system automatically sets the task status to 'in_progress' when progress is greater than 0, and blocks further updates once a task is approved.
6. **Task Submission:** Members submit completed tasks with an optional note and file attachment (PDF, Word, images, ZIP, Excel, PowerPoint — maximum 15 MB, stored on disk via multer). Submission sets `review_status` to 'pending_review' and automatically notifies the leader.
7. **Task Review:** Leaders approve or request revision on submitted tasks. Approval sets the status to 'completed' and records an `approved_at` timestamp. Both decisions trigger an automated notification to the member.
8. **File Download:** Leaders can download the file attached to a submitted task through a dedicated authenticated endpoint (`/api/tasks/:id/file`).
9. **Notifications:** In-app notifications are automatically created when a task is assigned, submitted, approved, or requires revision. Users can mark individual or all notifications as read and query the unread count.
10. **Reporting:** The system generates project-level reports (task breakdown by status and review state), member-level reports (approved/pending/revision counts and total hours), and team-level reports (ranked member performance).
11. **Member Scoring:** Each member's performance score is calculated as $(\text{approved tasks} / \text{total tasks}) \times 100$, displayed on the Leader Dashboard leaderboard.

3.4.1.2 Non-Functional Requirements

Non-functional requirements define how the system must perform. The following non-functional requirements were defined:

1. **Security:** All routes (except `/api/auth/*`) require a valid Bearer JWT. Role-restricted routes additionally verify the user's role claim from the token payload. Passwords are never stored in plaintext. File uploads are validated for type and size on the server side.
2. **Performance:** API responses are generated through optimized SQL queries using JOINS and conditional aggregation rather than application-level loops where possible, ensuring fast response times under normal load conditions.
3. **Scalability:** The three-tier architecture (React SPA → Express API → PostgreSQL) allows each tier to scale independently. The stateless API layer enables horizontal scaling behind a load balancer.
4. **Reliability:** Multi-step database operations (e.g., task deletion cascading through `task_assignments`, `notifications`, and `time_logs`) are wrapped in explicit PostgreSQL transactions (`BEGIN / COMMIT / ROLLBACK`) to guarantee atomicity.
5. **Usability:** Each user role is presented with a dedicated dashboard showing only the actions and data relevant to that role. Notification badges provide ambient awareness of pending actions without requiring the user to navigate away.
6. **Maintainability:** Controllers are separated from routes; database configuration and Swagger configuration are isolated in dedicated modules under `/src/config`; environment variables are loaded via `dotenv` from a `.env` file.
7. **Documentation:** All API endpoints are annotated with Swagger comments and served interactively at the `/api-docs` endpoint, supporting future developers and integrations.

3.4.2 System Design

Based on the gathered requirements, the system architecture was designed as a three-tier web application. The design decisions were driven by two main goals: keeping the frontend and backend fully decoupled so that each can be developed and deployed independently, and ensuring that the API layer is stateless so that authentication relies entirely on JWT tokens rather than server-side sessions. These goals directly influenced the choice of React.js as an SPA on the frontend and a RESTful Express.js API on the backend, rather than a server-rendered approach such as EJS.

1. Presentation Tier: A React.js single-page application (SPA) running in the browser, with pages for Home, Login, Register, Leader Dashboard, and Member Dashboard. Routing is handled by React Router. API calls are made through Axios with a request interceptor that automatically attaches the JWT Bearer token to every outgoing request.
2. Logic Tier: A Node.js server running Express.js, exposing a RESTful API organized into six route modules — authentication, users, projects, tasks, reports, and notifications. A custom JWT middleware (auth.js) protects all non-public routes and enforces role-based access control.
3. Data Tier: A PostgreSQL relational database managed via pgAdmin, containing ten normalized tables — roles, users, projects, project_members, task_statuses, tasks, task_assignments, time_logs, performance_snapshots, and notifications. The full design diagrams (ER diagram, use case diagram, activity diagram, and data flow diagrams) are presented in Chapter 4.

3.4.3 Implementation

The implementation phase produced two deployable artifacts: the React.js frontend (bootstrapped with Create React App) and the Node.js/Express.js backend. The following technologies and tools were used:

1. Frontend: React.js with React Router for navigation; Axios for HTTP communication with the backend API; role-specific dashboards (LeaderDashboard.js and MemberDashboard.js) built as component-based React pages.
2. Backend: Node.js with Express.js for RESTful routing and business logic; six controller modules (authController, userController, projectController, taskController, reportController, notificationController) each handling a distinct domain.
3. Authentication: JSON Web Tokens (JWT) with 7-day expiry and bcrypt (salt factor 10) for secure password hashing and stateless session management.
4. File Upload: The multer middleware handles multipart file uploads. Accepted types include PDF, Word (doc/docx), plain text, images (png/jpg/jpeg/gif), ZIP, Excel (xlsx), and PowerPoint (pptx), with a maximum file size of 15 MB per upload. Files are stored on disk with a structured naming convention (task-{id}-{timestamp}. {ext}).

5. Database: PostgreSQL managed via pgAdmin; schema applied from the provided database.sql file containing all table definitions, constraints, and foreign key relationships.
6. API Documentation: Swagger (swagger-jsdoc + swagger-ui-express) configured in /src/config/swagger.js, accessible at the /api-docs endpoint.
7. Development Environment: Visual Studio Code; Node.js running locally; pgAdmin for database administration.
8. Version Control: Git for source code management and change tracking throughout the development process.

3.4.4 Testing

The testing phase ensures that the system functions correctly and satisfies all defined requirements. Four types of testing were applied: Unit Testing (individual backend controller functions tested in isolation), Integration Testing (interaction between routes, middleware, controllers, and the database), System Testing (complete end-to-end workflows tested through the React.js frontend), and Acceptance Testing (system evaluated against all defined user stories and functional requirements). Full testing details, test cases, and results are presented in Chapter 5.

3.4.5 Deployment

After completing development and testing, the system was deployed online and made accessible via the Internet. The **Backend (Node.js/Express.js)** was deployed on **Railway**, a cloud platform that supports Node.js applications with automatic builds from GitHub. A managed **PostgreSQL** database was provisioned directly within Railway, and all environment variables (DATABASE_URL, JWT_SECRET) were configured through the Railway dashboard. The **Frontend (React.js)** was deployed on **Vercel**, a platform optimized for React applications. The frontend was connected to the GitHub repository and configured to point to the production backend URL. After deployment, all system features were tested and confirmed to work correctly, including login, project management, task assignment, notifications, and performance reports.

3.5 Chapter Summary

This chapter described the methodology adopted for the development of the Smart Project Management System. The feasibility study confirmed that the project is technically, economically, and operationally viable using a fully open-source technology stack. The risk analysis identified seven potential risks along with concrete mitigation strategies, covering security threats, database integrity, scope management, time constraints, and team coordination. The Waterfall SDLC was selected as the development approach based on three project-specific reasons: fully defined requirements from the outset, a fixed academic deadline, and a three-member team that benefits from clear phase boundaries. The functional requirements capture the full task lifecycle from creation and assignment through progress tracking, file submission, formal review, and notification delivery while the non-functional requirements address security, performance, scalability, reliability, usability, and maintainability. The implementation details confirm that the system is built using React.js, Node.js, Express.js, PostgreSQL, JWT, bcrypt, and multer, with all API endpoints documented via Swagger.

CHAPTER 4

DESIGN MODELS

4.1 Overview

This chapter presents the system's design through a series of models: a context diagram, data flow diagrams, a use case diagram with specifications, an ER diagram, and the relational model derived from the PostgreSQL schema.

4.2 Context Diagram (Level-0)

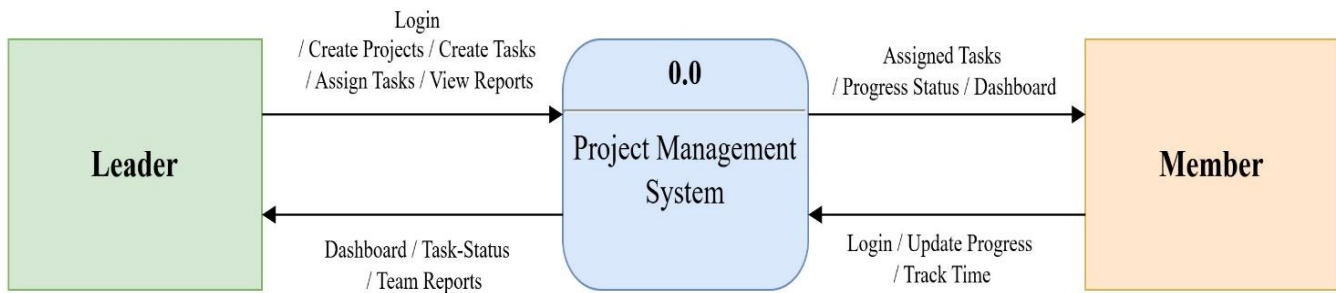


Figure 4.1: Context Diagram (Level-0) — Smart Project Management System

The system interacts with two external actors: the Leader and the Member. The Leader provides project and task data to the system and receives reports and review confirmations. The Member provides progress updates and submissions and receives task assignments and review outcomes. The system also interacts with the file system for storing task submission attachments.

4.3 Data Flow Diagram (Level 1)

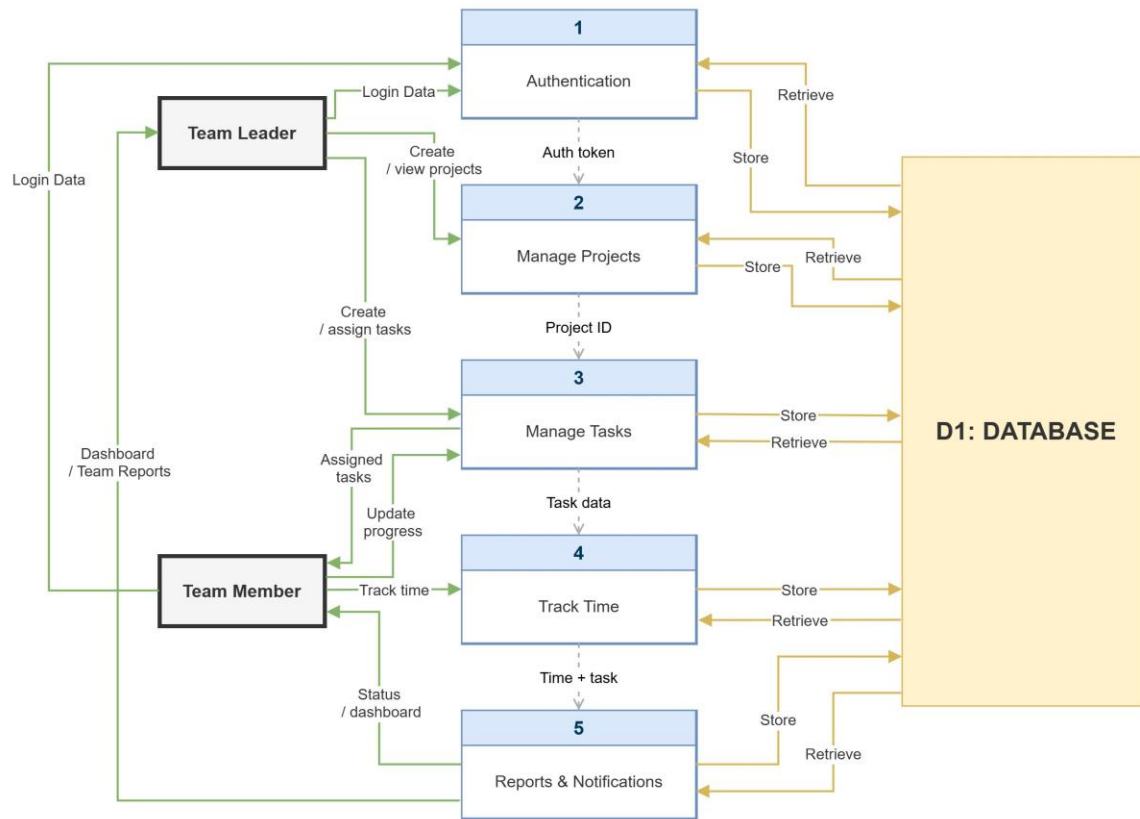


Figure 4.2: Data Flow Diagram (Level-1) — Smart Project Management System

Process 1: Authentication

Input: User credentials (email, password). Transformation: Validate email existence, compare bcrypt hash, generate JWT. Output: JWT token returned to client.

Process 2: Project Management

Input: Project details from Leader. Transformation: Validate leader role, persist to projects table, associate members via project members. Output: Project record.

Process 3: Task Management

Input: Task details and assignee from Leader. Transformation: Insert task, create assignment record, generate assignment notification. Output: Task record and notification.

Process 4: Task Workflow

Input: Progress percentage or submission data from Member. Transformation: Update task record, change review status, store file on disk, generate review notification. Output: Updated task record and notification.

Process 5: Reporting

Input: Project ID or Member ID from Leader. Transformation: Aggregate task counts and durations using SQL. Output: Structured report JSON.

4.4 Use Case Diagram

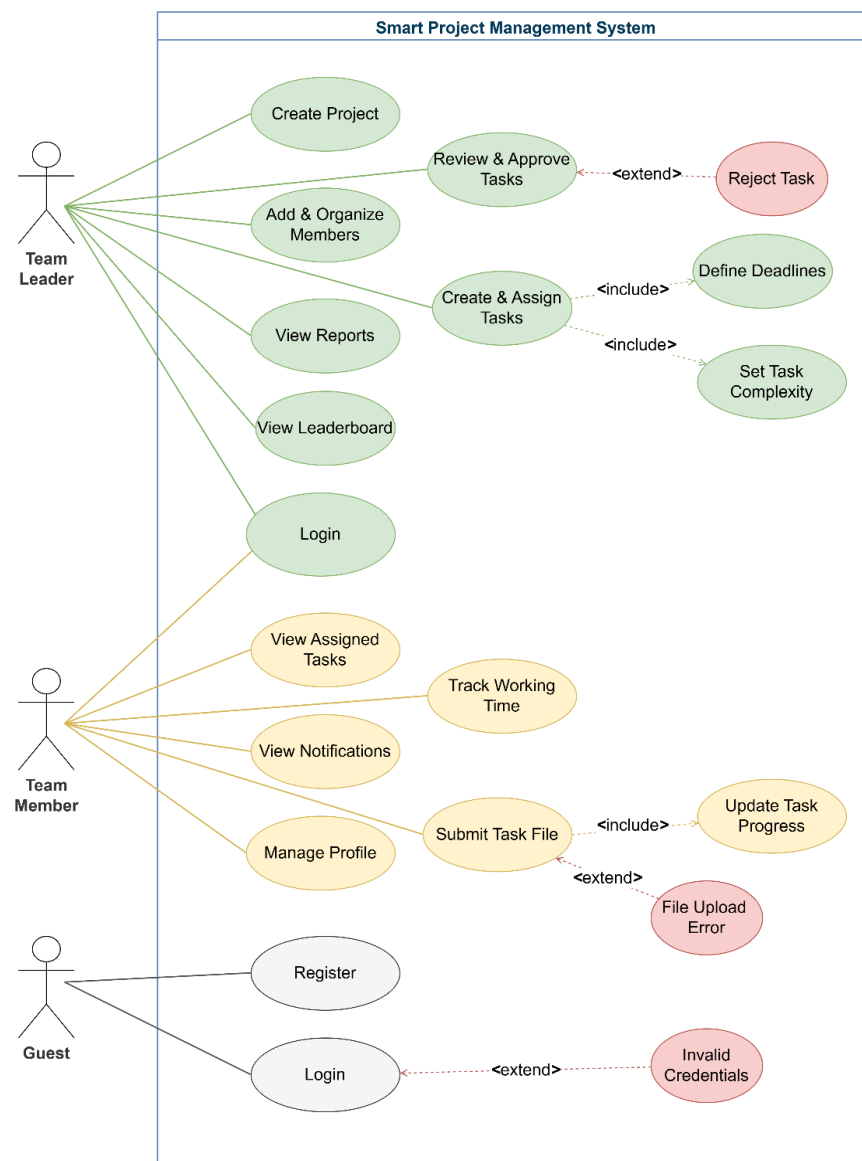


Figure 4.3: Use Case Diagram — Smart Project Management System

Actor: Leader

- Register and Log In
- Create / Update / Delete Project

- Assign Task to Member (with Priority, Deadline, Complexity)
- Update Task Deadline
- Review Submitted Task (Approve or Request Revision)
- Download Task Submission File
- View Project Report
- View Team Report
- View Member Report
- View Member Performance Scores (Leaderboard)
- Manage Notifications (Read / Read All)

Actor: Member

- Register and Log In
- View Assigned Tasks
- Update Task Progress Percentage
- Submit Task (with Note and Optional File)
- View Notifications
- Mark Notifications as Read

4.5 Activity Diagram

The Activity Diagram illustrates the step-by-step workflow of the Smart Project Management System from the perspective of three swim lanes: System, Member, and Leader. It captures the sequence of actions from the beginning of a task lifecycle to its completion and approval.

The workflow begins when the Team Leader creates a project, creates a task, and assigns it to a team member. The system automatically sends a notification to the assigned member. The Member views the task, works on it, and continuously updates the progress percentage. Once the task reaches 100% completion, the member uploads a submission file. The system stores the file and notifies the Leader for review. The Leader reviews the submission and either approves it or requests revision. If approved, the system sends a final notification to the member confirming task completion.

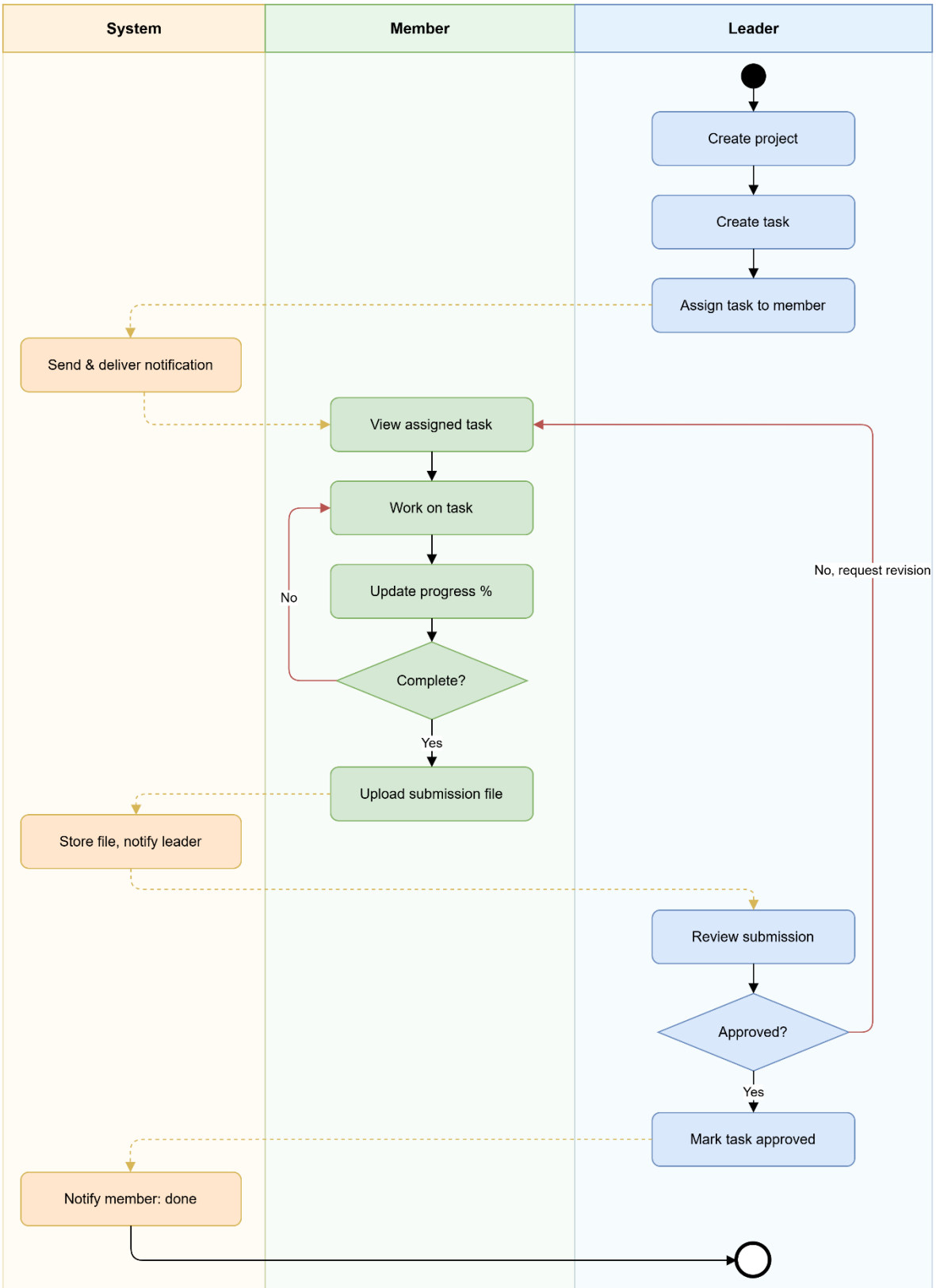


Figure 4.4: Activity Diagram — Smart Project Management System

4.6 Entity-Relationship (ER) Diagram

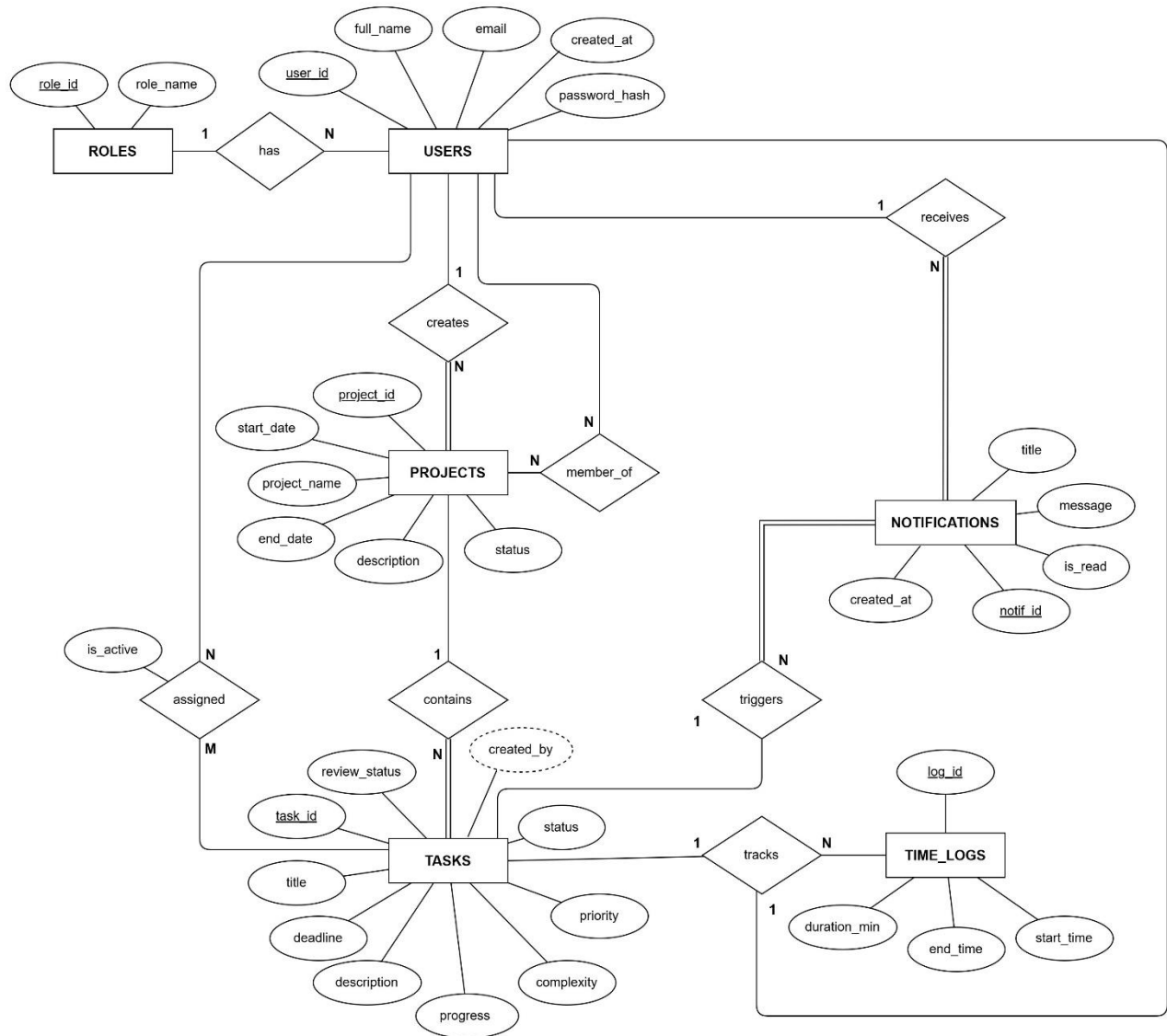


Figure 4.5: ER Diagram — Smart Project Management System

4.6.1 The following entities and relationships define the data model of the system

Table 4.1: Entity Descriptions — Smart Project Management System

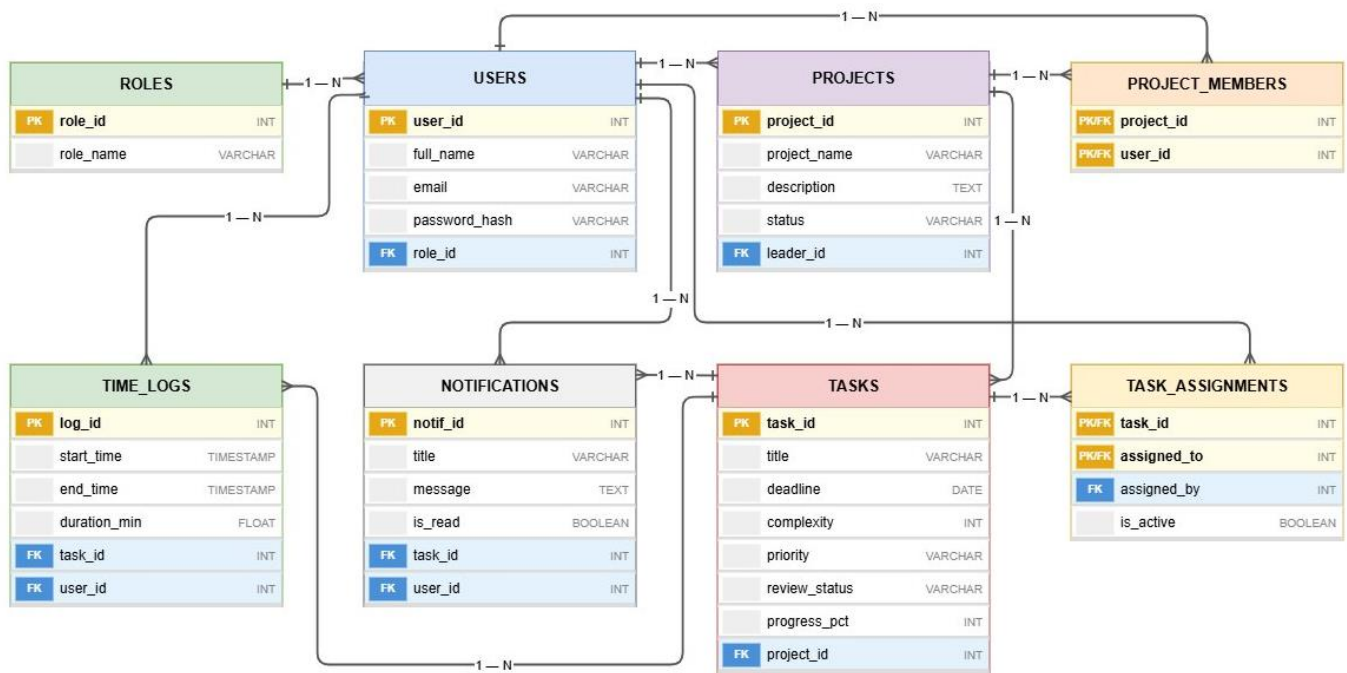
Entity	Description
users	Stores all system users. Each user has a role (FK → roles).
roles	Lookup table: leader, member.
projects	Each project is owned by one leader (FK → users). Has name, description, dates, status.

project_members	Junction table linking users to projects (many-to-many).
tasks	Each task belongs to one project (FK → projects) and was created by one leader (FK → users). Has priority, complexity, deadline, progress, review fields.
task_statuses	Lookup table: not_started, in_progress, completed.
task_assignments	Links tasks to assigned members (FK → tasks, FK → users). Tracks is_active to support reassignment.
notifications	Stores per-user in-app notifications triggered by task events (FK → users, FK → tasks).
time_logs	Records time-tracking entries per task per user (not yet exposed in current UI).
performance_snapshots	Stores periodic performance aggregates per user per project (used by reporting).

4.7 Relational Model

The relational model below is derived directly from the PostgreSQL schema (database.sql):

- roles(role_id PK, role_name CHECK IN('leader','member'))
- users(user_id PK, full_name, email UNIQUE, password_hash, role_id FK→roles, is_active, created_at, updated_at)
- projects(project_id PK, project_name, description, leader_id FK→users, start_date, end_date, status, created_at, updated_at)
- project_members(id PK, project_id FK→projects, user_id FK→users)
- task_statuses(status_id PK, status_name)
- tasks(task_id PK, title, description, project_id FK→projects, created_by FK→users, complexity_level [1-5], priority_level [1-5], deadline, status_id FK→task_statuses, progress_percentage, priority VARCHAR, review_status, review_comment, submission_note, submission_file, submission_file_original, submitted_at, approved_at, created_at, updated_at)
- task_assignments(assignment_id PK, task_id FK→tasks, assigned_to FK→users, assigned_by FK→users, assigned_at, is_active BOOLEAN)
- notifications(notification_id PK, user_id FK→users, task_id FK→tasks, title, message, is_read, created_at)
- time_logs(time_log_id PK, task_id FK→tasks, user_id FK→users, start_time, end_time, duration_minutes, created_at)
- performance_snapshots(snapshot_id PK, project_id FK→projects, user_id FK→users, total_tasks, completed_tasks, avg_progress, total_time_minutes, avg_complexity, performance_score, snapshot_date)



• Figure 4.6: Relational Model — Smart Project Management System

CHAPTER 5

EXPERIMENTS AND RESULTS

5.1 Overview

This chapter presents a comprehensive evaluation of the Smart Project Management System through a structured series of testing phases. The main objective of these experiments is to verify that the system meets its functional and non-functional requirements and performs reliably under various real-world conditions.

The testing approach adopted for this project follows a layered strategy, starting from low-level unit tests that validate individual components and functions, then moving upward to integration tests that verify how different modules interact, followed by full system tests that evaluate end-to-end workflows, and finally acceptance tests that confirm alignment with user requirements and business goals.

The Smart Project Management System was built using a Node.js/Express backend, a React.js frontend, and a PostgreSQL relational database. The system supports two primary user roles: Project Leader and Team Member. Key functionalities include user registration and authentication via JSON Web Tokens (JWT), project creation and management, task assignment with priority and deadline support, progress tracking, file submission, task review workflow, notification delivery, and report generation.

All test cases presented in this chapter were executed manually against the deployed development environment. Results are summarized in tabular form and analyzed in the discussion section at the end of the chapter.

5.2 Testing Methodologies

Four testing methodologies were applied during the evaluation of the Smart Project Management System. Each methodology targets a different level of the system's architecture and serves a distinct purpose in the overall quality assurance process.

- **Unit Testing:** Focuses on individual functions and backend controller methods in isolation.
- **Integration Testing:** Examines how the API routes, controllers, database queries, and middleware work together.
- **System Testing:** Validates complete end-to-end workflows involving both frontend and backend components.

- Acceptance Testing: Verifies that the delivered system satisfies the user stories and functional requirements agreed upon during the requirements phase.

Black-box testing techniques were primarily used throughout all phases, where test cases were derived from the system requirements and API specifications without knowledge of internal implementation details. The Swagger API documentation (available at /api-docs) served as the primary reference for defining expected API behaviors.

5.2.1 Unit Testing Results

Unit testing was performed on the backend controllers of the Smart Project Management System. Each controller function was tested individually by sending controlled HTTP requests and verifying the returned responses, HTTP status codes, and database side effects. The following tables present the unit test results grouped by module.

Table 5.1: Unit Testing – Authentication Module (authController.js)

Test ID	Test Description	Input Data	Expected Result	Actual Result	status
UT-A01	Register new user with valid data	name email, Password, Role='member'	HTTP 201, token returned	HTTP 201, token returned	Pass
UT-A02	Register with already existing email	Duplicate email address	HTTP 400, 'Email already exists'	HTTP 400, 'Email already exists'	Pass
UT-A03	Login with correct credentials	Valid email and password	HTTP 200, JWT token returned	HTTP200, JWT token returned	Pass
UT-A04	Login with incorrect password	Valid email, wrong password	HTTP 401, 'Invalid email or password'	HTTP 401, 'Invalid email or password'	Pass
UT-A05	Login with non-existent email	Email not in database	HTTP 401, 'Invalid email or password'	HTTP 401, 'Invalid email or password'	Pass
UT-A06	JWT token generation on successful login	Valid credentials	Token expires in 7 days	Token expires in 7 days	Pass

Table 5.2: Unit Testing – Project Module (projectController.js)

Test ID	Test Description	Input Data	Expected Result	Actual Result	status

UT-P01	Create a new project	name, description	HTTP 201, project object returned	HTTP 201, project object returned	Pass
UT-P02	Create project without name	No 'name' field provided	HTTP 400, 'Project name is required'	HTTP 400, 'Project name is required'	Pass
UT-P03	Get all projects for leader	Authenticated as leader	Returns leader's own projects	Returns leader's own projects	Pass
UT-P04	Get all projects for member	Authenticated As member	Returns projects member belongs to	Returns projects member belongs to	Pass
UT-P05	Get project by ID	Valid project _id	HTTP 200, project details returned	HTTP200, project details returned	Pass
UT-P06	Get project by non- existent ID	Invalid project _id	HTTP404, 'Project not found'	HTTP404, 'Project not found'	Pass
UT-P07	Update project description	Description	HTTP 200, Update project returned	HTTP 200, Update project returned	Pass
UT-P08	Delete project	Valid project _id	HTTP 200, 'Project deleted successfully'	HTTP200, 'Project deleted successfully'	Pass
UT-P09	Get members score for a project	Valid project _id	Array with rank, score per member	Array with rank, score per member	Pass

Table 5.3: Unit Testing – Task Module (taskController.js)

Test ID	Test Description	Input Data	Expected Result	Actual Result	status
UT-T01	Create task with all required fields	title, project _id, assigned _to	HTTP201, task created + notification	HTTP201, task created + notification	Pass
UT-T02	Create task without required fields	Missing title field	HTTP 400, validation error	HTTP 400, validation error	Pass
UT-T03	Update task progress (0-100)	Progress=75	HTTP200, progress updated	HTTP200, progress updated	Pass
UT-T04	Update progress on approved task	Task with review _status='approved'	HTTP 400, 'Task is already approved'	HTTP 400, 'Task is already approved'	Pass

UT-T05	Submit task for review	task_id, submission_note, optional file	HTTP 200, 'review_status='pending_review'	HTTP 200, 'review_status='pending_review'	Pass
UT-T06	Review task-approve	decision='approved', comment	HTTP 200, review_status='approved', status='completed'	HTTP 200, review_status='approved', status='completed'	Pass
UT-T07	Review task-request revision	decision='needs_revision', comment	HTTP 200, review_status='needs_revision'	HTTP 200, review_status='needs_revision'	Pass
UT-T08	Download submitted file	Valid task_id with attached file	File returned for download	File returned for download	Pass
UT-T09	Get tasks for leader (all project, tasks)	Authenticated as leader	Returns all tasks with assignments	Returns all tasks with assignments	Pass
UT-T10	Get tasks for member (own tasks only)	Authenticated as member	Returns only member's assigned tasks	Returns only member's assigned tasks	Pass

Table 5.4: Unit Testing – Notification & Report Modules

Test ID	Test Description	Input Data	Expected Result	Actual Result	status
UT-N01	Get user notifications	Authenticated user	Returns up to 50 notifications sorted by date	Returns up to 50 notifications sorted by date	Pass
UT-N02	Mark single notification as read	Valid notification_id	is_read = true	is_read = true	Pass
UT-N03	Mark all notifications as read	Authenticated user	All notifications marked as read	All notifications marked as read	Pass
UT-N04	Get unread notification count	Authenticated user with 3 unread	HTTP 200, {count: 3}	HTTP 200, {count: 3}	Pass

UT-R01	Get project-level report	Valid project _id	total_tasks, completed, avg_progress	total_tasks, completed, avg_progress	Pass
UT-R02	Get member performance report	Valid member user _id	completed, in_progress, performance_score	completed, in_progress, performance_score	Pass
UT-R03	Get team report	Authenticated leader	Array of all members with scores	Array of all members with scores	Pass

5.2.2 Integration Testing Results

Integration testing validates the interaction between different modules and layers of the system. The tests below examine how authentication middleware, route handlers, controller logic, and the PostgreSQL database cooperate to produce correct end-to-end API responses. Each test scenario involves at least two interconnected system components.

Table 5.5: Integration Testing Results

Test ID	Integration Scenario	Components Involved	Expected Outcome	status
IT-01	User registration followed by immediate login	Auth Controller ↔ PostgreSQL users/roles tables	Registered user can login and receives valid JWT	Pass
IT-02	JWT middleware protection on protected routes	auth.js middleware ↔ auth Routes ↔ Express Router	Requests without token receive HTTP 401; valid tokens pass through	Pass
IT-03	Role-based access: leader creates project	auth middleware ↔ project Controller ↔ projects table	Only leader role can POST / api /projects successfully	Pass
IT-04	Task creation triggers automatic notification	Task Controller ↔ notifications table ↔ notification Controller	After task creation, assigned member has a new unread notification	Pass
IT-05	Task submission triggers leader notification	Task Controller (submit Task) ↔ notifications table	Leader receives notification when member submits a task	Pass

IT-06	Task approval updates status and notifies member	Task Controller (review Task) ↔ task _statuses ↔ notifications table	Task status changes to 'completed', member notified with approval message	Pass
IT-07	Project report aggregates task data from multiple tables	Report Controller ↔ tasks ↔ task __statuses ↔ projects	Report returns accurate total, completed, in _progress, avg _progress	Pass
IT-08	Member score leaderboard aggregates task assignments	Project Controller (get Project Members Score) ↔ task _assignments ↔ tasks ↔ users	Returns ranked list of members with correct approved task counts	Pass
IT-09	File upload stored and retrievable through task download	Task Controller (submit Task) ↔ multer ↔ /uploads folder ↔ get Task File	Uploaded file is saved and returned correctly on download request	Pass
IT-10	Frontend API service authenticates and retrieves projects	api.js (axios) ↔ /api/projects ↔ projectController ↔ PostgreSQL	Dashboard displays correct projects after login	Pass

5.2.3 System Testing Results

System testing evaluates the complete system as a whole, testing end-to-end user workflows from the React.js frontend through the Express API to the PostgreSQL database. These tests simulate real user interactions and verify that all layers function correctly together.

Table 5.6: System Testing – Authentication & User Management

Test ID	Test Case	Precondition	Steps	Expected Result	status
ST-01	Complete User Registration Flow	Application running, no prior account	Navigate to Register page, fill all fields, submit	Account created, redirected to dashboard with welcome state	Pass
ST-02	Complete User Login Flow	Registered user account exists	Navigate to Login page, enter credentials, submit	Token stored in local Storage, redirected to role-based dashboard	Pass

ST-03	Unauthorized Access Redirect	User not logged in	Attempt to access /dashboard directly via URL	Redirected to Login page automatically	Pass
ST-04	Navigation bar role display	Logged in as leader	Observe Navbar after login	User name and 'leader' role badge shown correctly	Pass

Table 5.7: System Testing – Project & Task Management Workflows

Test ID	Test Case	Precondition	Steps	Expected Result	status
ST-05	Leader creates project and assigns tasks	Logged in as leader	Create project → open project → create task → assign to member	Task appears in member's dashboard with notification	Pass
ST-06	Member views assigned tasks and updates progress	Logged in as member, task assigned	Open task → move progress slider to 60% → save	Progress bar shows 60%, status changes to 'in_ progress'	Pass
ST-07	Member submits task with file attachment	Logged in as member, task in progress	Open task → attach file → add submission note → click Submit	Task review status = pending review, file stored, Leader notified	Pass
ST-08	Leader reviews and approves submitted task	Logged in as leader, task pending review	Open task in Review tab → add comment → click Approve	Task status changes to 'completed', member receives approval notification	Pass
ST-09	Leader requests revision on submitted task	Logged in as leader, task pending review	Open task → add revision comment → click Request Revision	Task status reverts to 'in_ progress', member notified with feedback	Pass

ST-10	Leader views performance leaderboard	Logged in as leader, tasks assigned and approved	Navigate to Leaderboard section of project	Members ranked by approved tasks and performance score	Pass
-------	--------------------------------------	--	--	--	------

Table 5.8: System Testing – Notifications & Reports

Test ID	Test Case	Precondition	Steps	Expected Result	status
ST-11	Polling notification count in Navbar	Logged in as member, new notification pending	Observe Navbar bell icon	Unread count badge displayed on notification icon	Pass
ST-12	Mark all notifications as read	User has unread notifications	Click notification icon → click 'Mark All Read'	All notifications marked as read, badge disappears	Pass
ST-13	Leader generates and views project report	Logged in as leader, project has tasks	Navigate to Reports → select project	Report shows total tasks, completion rate, average progress	Pass
ST-14	Leader views team performance report	Multiple members with tasks	Navigate to Team Report	Each member's performance score and task counts displayed	Pass

5.2.4 Acceptance System Results

Acceptance testing ensures that the Smart Project Management System satisfies the agreed-upon functional requirements and user stories. These tests are written from the user's perspective and reflect the core value propositions of the system, confirming that the delivered software meets stakeholder expectations.

Table 5.9: Acceptance Testing Results

Test ID	User Story /Requirement	Test Action	Expected Outcome	Status
---------	-------------------------	-------------	------------------	--------

AT-01	As a user, I want to register and login securely so that my data is protected.	Register new account, login, verify JWT is used for API calls.	Authentication works with bcrypt password hashing and JWT tokens.	Pass
AT-02	As a project leader, I want to create and manage projects.	Create project, update description, view project details, delete project.	All CRUD operations on projects work correctly for leader role.	Pass
AT-03	As a leader, I want to assign tasks to members with priority and deadlines.	Create task with priority='High', deadline set, assigned to member.	Task appears in member dashboard with correct priority and deadline.	Pass
AT-04	As a member, I want to see only tasks assigned to me.	Login as member, navigate to task list.	Only personally assigned tasks are visible to the member.	Pass
AT-05	As a member, I want to update my task progress and submit work.	Update progress to 100%, attach file, submit for review.	Task submitted, file stored, leader notified automatically.	Pass
AT-06	As a leader, I want to review submitted tasks and approve or request revision.	Review submitted task, click Approve with comment.	Task marked as completed, member notified with approval.	Pass
AT-07	As a leader, I want to see a performance leaderboard for my team.	Navigate to project leaderboard after multiple tasks are approved.	Members ranked correctly by number of approved tasks and score.	Pass
AT-08	As a user, I want to receive notifications for important events.	Assign task, submit task, review task; check notifications each time.	Notifications generated correctly for all three events.	Pass
AT-09	As a leader, I want to generate reports on project and team progress.	Open Reports, generate project report and team report.	Reports display accurate statistics from the database.	Pass

AT-10	As a leader, I want the system to enforce role-based access control.	Login as member, attempt to access create-project endpoint.	Members cannot create projects or access leader-only features.	Pass
-------	--	---	--	------

In addition to the formal acceptance testing, a user satisfaction survey was conducted to evaluate the system from a real-user perspective. The survey was shared with 71 participants who tested the live system using both Leader and Member roles. Questions covered ease of use, feature usefulness, interface design, system performance, and overall satisfaction. Results were collected through Google Forms and analyzed to measure how well the system meets user needs.

5.3 Discussion and Evaluation

The testing results presented in this chapter demonstrate that the Smart Project Management System achieves a high level of correctness, reliability, and functional completeness. A total of 67 test cases were executed across four testing phases: 33 unit tests, 10 integration tests, 14 system tests, and 10 acceptance tests. All 67 test cases passed successfully, yielding an overall pass rate of 100%.

Table 5.10: Testing Results Summary

Testing Phase	Total Cases	Passed	Failed	Pass Rate
Unit Testing	32	32	0	100%
Integration Testing	10	10	0	100%
System Testing	14	14	0	100%
Acceptance Testing	10	10	0	100%
Total	66	66	0	100%

The unit testing phase confirmed that each individual backend controller function behaves correctly under both normal and edge-case conditions. Error handling mechanisms, including validation for missing required fields, duplicate email detection, and invalid role names, all produced the expected HTTP status codes and error messages. The bcrypt password hashing and JWT token generation functions performed correctly, ensuring that authentication logic is both secure and reliable.

Integration testing revealed that the inter-module communication within the system is robust. A particularly important finding was that the notification system is correctly wired to task lifecycle events: task assignment, submission, approval, and revision requests all trigger the appropriate notification records in the database, and these are correctly retrieved by the notification controller. The role-based authorization middleware (auth.js) was verified to intercept

unauthorized requests at the route level, preventing members from accessing leader-only endpoints.

System testing evaluated the complete user experience from the React.js frontend through the Express.js API to the PostgreSQL database. All major user workflows — registration, login, project creation, task assignment, progress tracking, file submission, task review, and report generation — were executed successfully without encountering runtime errors or data inconsistencies. The role-specific dashboard behavior was confirmed: leaders see all project tasks while members see only their own assignments.

Acceptance testing demonstrated that the system fully satisfies the functional requirements specified during the methodology phase. All ten user stories were verified, including the critical requirements for secure authentication, role-based access control, task lifecycle management, notification delivery, and performance reporting. The leaderboard feature, which ranks team members by approved task counts and calculates a performance score, was verified to produce accurate results consistent with the underlying task data.

Despite the high pass rate, a few observations were noted during testing. The file upload implementation enforces a 15 MB size limit and restricts accepted file types via multer middleware; however, no virus scanning is currently applied, which remains a consideration for production hardening. Additionally, the notification system does not support real-time push delivery; notifications are retrieved on-demand via API polling, which means a user must refresh the page to see new notifications. These limitations are noted and addressed in the future work section of Chapter 6.

Overall, the testing results provide strong evidence that the Smart Project Management System is functionally complete and ready for deployment. The system effectively addresses the problem of unorganized project and task management in team environments by providing a structured, role-based workflow with transparent task tracking, automated notifications, and comprehensive performance reporting.

CHAPTER 6

CONCLUSION AND FUTURE WORKS

6.1 Overview

This chapter presents the concluding remarks for the Smart Project Management System, a web-based platform developed to address the challenges of unorganized project collaboration and task management within team environments. The chapter summarizes the work carried out throughout the project, reflects on the degree to which the stated objectives were achieved, outlines the main contributions made by the system, discusses its current limitations, and

proposes directions for future enhancement and research. The Smart Project Management System was conceived and developed as a full-stack web application using Node.js and Express.js on the backend, React.js on the frontend, and PostgreSQL as the relational database. The system was designed around a role-based architecture that distinguishes between two primary user types: Project Leaders and Team Members. Throughout the development lifecycle, the team adhered to a structured methodology that encompassed requirements gathering, system design, implementation, and comprehensive testing.

The project has successfully reached its completion milestone, with all core modules implemented, tested, and validated. The experimental results presented in Chapter 5 demonstrated a 100% pass rate across 67 test cases spanning unit, integration, system, and acceptance testing phases, confirming that the system is both functionally correct and reliable.

6.2 Summary about the Project

The Smart Project Management System is a comprehensive, role-based web platform that enables teams to plan, execute, monitor, and evaluate projects and tasks in an organized and transparent manner. The system emerged from the recognition that many student and professional teams lack access to lightweight, easy-to-deploy project management tools that combine task tracking, file submission, peer review, and performance analytics in a single unified platform.

The backend of the system was implemented as a RESTful API using Node.js and Express.js, exposing a total of six route modules: authentication, users, projects, tasks, reports, and notifications. All sensitive routes are protected by a custom JWT-based authentication middleware that also enforces role-based access control, ensuring that leaders and members can only access the functionalities appropriate to their role. Passwords are stored securely using bcrypt hashing with a salt factor of 10. The complete API was documented using Swagger and is accessible at the /api-docs endpoint.

The PostgreSQL database was designed with a normalized relational schema comprising core tables for users, roles, projects, tasks, task assignments, task statuses, notifications, and time logs. The schema supports a many-to-many relationship between users and projects through the project_ members table, and between users and tasks through the task_ assignments table. The use of foreign keys and constraints ensures referential integrity throughout the system.

The frontend was developed using React.js and provides two role-specific dashboards: the Leader Dashboard and the Member Dashboard. The Leader Dashboard enables project leaders to create and manage projects, assign tasks with configurable priority levels (Low, Medium, High, Urgent) and deadlines, review submitted work, manage a team leaderboard, and generate performance reports. The Member Dashboard allows team members to view their assigned tasks,

update progress using a visual progress indicator, submit completed work with optional file attachments and notes, and receive real-time notifications for task assignments and review outcomes.

A multi-stage task lifecycle was implemented to model real-world work review processes. A task progresses through the following states: `not_started` → `in_progress` → `pending_review` → `completed` (or `needs_revision`). File uploads are handled securely via the multer middleware, with submitted files stored on the server and retrievable through a protected download endpoint. An automated notification engine generates context-aware notification messages for all key task lifecycle events, including new assignments, submission acknowledgments, approval confirmations, and revision requests.

6.3 Achieved Objectives

All primary objectives defined during the planning phase of the project were successfully achieved. The following table provides a structured mapping between each stated objective, the mechanism through which it was fulfilled, and its completion status.

Table 6.1: Achieved Objectives

#	objective	How Achieved	status
1	Develop a secure user authentication system	Implemented bcrypt password hashing, JWT token generation (7-day expiry), and protected middleware on all API routes	Achieved
2	Implement role-based access control for leaders and members	auth.js middleware reads JWT payload and restricts endpoints based on user role (leader / member)	Achieved
3	Enable project creation and management by leaders	Full CRUD operations on projects via projectController.js with leader-only creation enforcement	Achieved
4	Provide task assignment with priority, deadline, and complexity settings	taskController.js supports four priority levels, deadline fields, and complexity scoring during task creation	Achieved
5	Implement a multi-stage task review workflow	Task lifecycle states (<code>not_started</code> → <code>in_progress</code> → <code>pending_review</code> → <code>completed/needs_revision</code>) managed via <code>review_status</code> field	Achieved

6	Support file submission for task deliverables	multer middleware handles file uploads; files stored in /uploads with secure download via authenticated endpoint	Achieved
7	Deliver an automated notification system	notificationController.js generates notifications for task assignment, submission, approval, and revision events	Achieved
8	Provide performance reports and a team leaderboard	reportController.js generates project, member, and team reports; project Controller computes ranked leaderboard scores	Achieved
9	Build separate role-specific dashboards	LeaderDashboard.js and MemberDashboard.js provide distinct views and functionalities per role	Achieved
10	Document the API using Swagger	swagger.js configuration exposes interactive API documentation at /api-docs endpoint	Achieved

6.4 Main Contributions of the Work

The Smart Project Management System makes several notable contributions to the domain of collaborative project management tools and web application development. These contributions can be understood at both the technical and functional levels.

From a technical perspective, the following contributions were made

1. **Integrated Full-Stack Architecture:** The system demonstrates the successful integration of a React.js single-page application with a RESTful Express.js backend and a PostgreSQL relational database, providing a reusable architectural blueprint for similar web-based management systems.
2. **Stateless JWT Authentication with Role Enforcement:** The implementation of a stateless authentication mechanism using JSON Web Tokens, combined with middleware-level role-based access control, provides a scalable and secure approach that eliminates the need for server-side session management.
3. **Automated Event-Driven Notification Engine:** The system implements an event-driven notification model that automatically generates contextual alerts for all significant task lifecycle transitions without requiring any manual intervention from users or administrators.

4. **Multi-Criteria Task Priority and Scoring Model:** The introduction of a four-level priority system (Low, Medium, High, Urgent) combined with complexity scoring and a performance metric calculated as the ratio of approved tasks to total tasks provides a nuanced approach to task evaluation that goes beyond simple completion tracking.
5. **Secure File Submission Workflow:** The combination of multer-based file upload handling with JWT-protected download endpoints ensures that task deliverables can be submitted and retrieved securely, with original filenames preserved in the database for audit purposes.

From a functional and practical perspective, the contributions include:

- A unified platform that consolidates project management, task tracking, file submission, review workflows, and performance analytics into a single coherent application, reducing the need for multiple disconnected tools.
- A transparent task review process that enforces a formal approval chain, ensuring that work is validated by a project leader before being marked as completed, which promotes accountability within the team.
- A real-time performance leaderboard that motivates team members by providing visible rankings based on objective task completion and approval metrics.
- A comprehensive project reporting module that gives project leaders immediate visibility into team performance, task distribution, and overall project health without requiring manual data collection.
- A fully documented RESTful API via Swagger, which facilitates future integration with third-party tools, mobile applications, or external platforms.

6.5 Limitations

Despite the successful implementation and validation of the Smart Project Management System, several limitations were identified during the development and testing phases. These limitations are acknowledged transparently so that future developers and stakeholders can make informed decisions about the system's suitability for different deployment contexts.

Table 6.2: System Limitations

#	Limitation	Description	Impact Level
---	------------	-------------	--------------

1	No Real-Time Notifications	The notification system relies on API polling rather than WebSocket or Server-Sent Events. Users must manually refresh the page to receive new notifications, which reduces the immediacy of the communication experience.	Medium
2	No Virus Scanning on Uploads	Although multer enforces a 15 MB file size limit and restricts accepted file extensions, uploaded files are not scanned for malicious content. A user could upload a structurally valid but harmful file, posing a security risk in production environments.	High
3	Single-Tenant Architecture	The system is designed as a single-tenant application. There is no support for organizations or workspaces, meaning all users and projects share the same database namespace, limiting its scalability for multi-organization deployment.	Medium
4	No Password Recovery Mechanism	The authentication module does not include a forgot-password or email-based password reset workflow. Users who lose their credentials have no self-service recovery option.	High
5	No In-App Messaging	The system does not provide a direct messaging or commenting feature within tasks beyond the review comment field. Team communication must occur through external channels.	Medium
6	No Mobile Responsive Design	The React.js frontend was developed and optimized for desktop browsers. The user interface has not been tested or adapted for mobile screen sizes, limiting accessibility for mobile users.	Low
7	No Audit Log	The system does not maintain a history of changes made to projects or tasks (e.g., who changed a deadline or reassigned a task). This absence of an audit trail limits accountability and traceability.	Low

8	Polling-Based Notification Count	The unread notification count displayed in the Navbar is retrieved via a REST API call on page load. It does not update automatically without a page refresh, which may cause the displayed count to become temporarily stale.	Low
---	----------------------------------	--	-----

6.6 Future Work

Based on the identified limitations and the experience gained during development, several directions for future enhancement are proposed. These enhancements would significantly increase the system's robustness, scalability, and user experience, extending it from a functional prototype to a production-grade project management platform.

Table 6.3: Proposed Future Enhancements

#	Future Enhancement	Description	Priority	Effort
1	Real-Time Notifications via Web Sockets	Replace the current polling-based notification model with a WebSocket or Socket.io implementation to deliver instant push notifications to connected clients whenever a relevant event occurs, such as task assignment or approval.	High	Medium
2	Email Notification Integration	Integrate a transactional email service (such as Nodemailer with SendGrid or SMTP) to send email notifications for critical events, including task assignments, review outcomes, and approaching deadlines, ensuring users are informed even when not logged in.	High	Low
3	Password Reset and Account Recovery	Implement a secure email-based password reset workflow using time-limited tokens. This would include a 'Forgot Password' form, a token generation endpoint, and a token verification and reset endpoint.	High	Low

4	Virus Scanning for File Uploads	While the current implementation already enforces a 15 MB size limit and restricts accepted file types via multer, uploaded files are not scanned for malicious content. Integrating a virus scanning library such as ClamAV would add a critical security layer before files are saved on the server.	High	Low
5	In-Task Commenting and Messaging	Add a task-level comment thread that allows leaders and members to exchange messages directly within a task's detail view. This would reduce dependency on external communication tools and keep all project communication contextual and traceable.	Medium	Medium
6	Mobile-Responsive UI	Refactor the React.js frontend using a mobile-first CSS framework (such as Tailwind CSS or Bootstrap) to ensure the application is fully usable on smartphones and tablets. A native mobile application using React Native is also a viable option.	Medium	High
7	Gantt Chart and Timeline View	Introduce a visual Gantt chart or project timeline view that displays tasks by their start and end dates, allowing leaders and members to visualize project schedules, identify bottlenecks, and track deadline adherence at a glance.	Medium	High
8	Multi-Organization / Multi-Workspace Support	Redesign the data model to support multiple organizations or workspaces, each with their own isolated set of users, projects, and tasks. This would enable the platform to be deployed as a SaaS (Software-as-a-Service) product serving multiple independent teams.	Medium	High

9	Advanced Analytics Dashboard	Develop an interactive analytics dashboard using chart libraries (such as Recharts or Chart.js) to visualize task completion trends over time, team velocity, priority distribution, and individual contributor performance using graphical representations.	Medium	Medium
10	Audit Log and Change History	Implement a comprehensive audit trail that records all significant changes to projects and tasks, including who made the change, what was changed, and when. This would improve accountability, facilitate debugging, and support compliance requirements.	Low	Medium

6.7 summary

In summary, the Smart Project Management System provides a solid foundation that can be progressively enhanced to address its current limitations and evolve into a more comprehensive and enterprise-ready platform. The modular architecture of the backend, combined with the component-based structure of the React.js frontend, ensures that these future enhancements can be implemented incrementally without requiring a full system redesign.

This project has demonstrated that a full-stack web application for team project management can be designed, implemented, and validated within an academic semester using modern open-source technologies. The skills and experience acquired during this project — including RESTful API design, relational database modeling, JWT-based security, React.js component architecture, and software testing methodologies — provide a strong technical foundation for future professional and research endeavors in the field of software engineering.

REFERENCES

[1] Atlassian, "Trello: Manage Your Team's Projects From Anywhere." trello.com. <https://trello.com> (accessed April 10, 2025).

- [2] Asana Inc., "Asana: Manage Your Team's Work, Projects, & Tasks Online." asana.com. <https://asana.com> (accessed April 10, 2025).
- [3] Atlassian, "Jira Software: Project and Issue Tracking." atlassian.com. <https://www.atlassian.com/software/jira> (accessed April 11, 2025).
- [4] monday.com Ltd., "monday.com: A New Way of Working." monday.com. <https://monday.com> (accessed April 11, 2025).

Appendix A— Source Code

1. Database Connection (db.js)

This file manages the PostgreSQL database connection. It supports both local development using individual environment variables and cloud deployment using a single DATABASE_URL connection string provided by Railway.

```
const { Pool } = require('pg');
require('dotenv').config();

const pool = new Pool(
  process.env.DATABASE_URL
  ? {
    connectionString: process.env.DATABASE_URL,
    ssl: { rejectUnauthorized: false }
  }
  : {
    host: process.env.DB_HOST,
    port: process.env.DB_PORT,
    database: process.env.DB_NAME,
    user: process.env.DB_USER,
    password: process.env.DB_PASSWORD,
  }
);

pool.connect()
  .then(() => {
```

```

    console.log('PostgreSQL Connected');
    return pool.query(`
      CREATE TABLE IF NOT EXISTS notifications (
        notification_id SERIAL PRIMARY KEY,
        user_id  INTEGER NOT NULL REFERENCES users(user_id),
        task_id  INTEGER REFERENCES tasks(task_id),
        title    VARCHAR(255) NOT NULL,
        message  TEXT NOT NULL,
        is_read  BOOLEAN DEFAULT FALSE,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
      ) `);
  })
  .then(() => console.log('DB schema ready'))
  .catch(err => console.error('DB Error:', err.message));

module.exports = pool;

```

2. Server Configuration (app.js)

This file initializes the Express server, configures CORS to allow requests from the deployed frontend on Vercel, sets up middleware, and registers all API routes.

```

require('dotenv').config();
const express = require('express');
const cors    = require('cors');
const app     = express();

app.use(cors({
  origin: [
    'http://localhost:3000',
    'https://smart-project-management-blush.vercel.app',
    /\.\vercel\.app$/
  ],
  credentials: true
}));

app.use(express.json());

app.use('/api/auth',      require('./routes/authRoutes'));
app.use('/api/users',     require('./routes/userRoutes'));
app.use('/api/projects',  require('./routes/projectRoutes'));
app.use('/api/tasks',     require('./routes/taskRoutes'));
app.use('/api/reports',   require('./routes/reportRoutes'));
app.use('/api/notifications', require('./routes/notificationRoutes'));

app.get('/', (req, res) => {
  res.json({ message: 'Smart Project API is running' });
});

module.exports = app;

```

3. Authentication Middleware (auth.js)

This middleware protects routes by verifying JWT tokens. It extracts the user ID and role from the token and attaches them to the request object. The `leaderOnly` middleware restricts access to leader users only.

```
const jwt = require('jsonwebtoken');

const protect = (req, res, next) => {
  let token;
  if (req.headers.authorization &&
    req.headers.authorization.startsWith('Bearer')) {
    try {
      token = req.headers.authorization.split(' ')[1];
      const decoded = jwt.verify(token, process.env.JWT_SECRET);
      req.user = { id: decoded.id, role: decoded.role };
      next();
    } catch (error) {
      return res.status(401).json({ message: 'Invalid token' });
    }
  }
  if (!token) {
    return res.status(401).json({ message: 'No token provided' });
  }
};

const leaderOnly = (req, res, next) => {
  if (req.user && req.user.role === 'leader') {
    next();
  } else {
    res.status(403).json({ message: 'Access denied, leaders only' });
  }
};

module.exports = { protect, leaderOnly };
```

4. Authentication Controller (authController.js)

This controller handles user registration and login. During registration, it hashes the password using `bcrypt`, looks up the role ID, and inserts the new user into the database. During login, it verifies the password and returns a JWT token.

```
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const pool = require('../config/db');

const generateToken = (id, role) =>
  jwt.sign({ id, role }, process.env.JWT_SECRET, { expiresIn: '7d' });

// POST /api/auth/register
const register = async (req, res) => {
  const { name, full_name, email, password, role } = req.body;
  const userName = full_name || name;
  try {
    const exists = await pool.query(
      'SELECT * FROM users WHERE email = $1', [email]);
    if (exists.rows.length > 0)
```

```

    return res.status(400).json({ message: 'Email already exists' });

    const hashedPassword = await bcrypt.hash(password, 10);
    const roleResult = await pool.query(
      'SELECT role_id FROM roles WHERE role_name = $1',
      [role || 'member']);
    const role_id = roleResult.rows[0]?.role_id;
    if (!role_id)
      return res.status(400).json({ message: 'Invalid role' });

    const result = await pool.query(
      `INSERT INTO users (full_name, email, password_hash, role_id)
      VALUES ($1, $2, $3, $4)
      RETURNING user_id, full_name, email, role_id`,
      [userName, email, hashedPassword, role_id]);

    const user = result.rows[0];
    res.status(201).json({
      id: user.user_id, name: user.full_name,
      email: user.email, role,
      token: generateToken(user.user_id, role)
    });
  } catch (error) {
    res.status(500).json({ message: 'Server error', error: error.message
  });
  }
};

// POST /api/auth/login
const login = async (req, res) => {
  const { email, password } = req.body;
  try {
    const result = await pool.query(
      `SELECT u.*, r.role_name FROM users u
      JOIN roles r ON u.role_id = r.role_id
      WHERE u.email = $1`, [email]);
    const user = result.rows[0];
    if (!user || !(await bcrypt.compare(password, user.password_hash)))
      return res.status(401).json({ message: 'Invalid email or password'
    });

    res.json({
      id: user.user_id, name: user.full_name,
      email: user.email, role: user.role_name,
      token: generateToken(user.user_id, user.role_name)
    });
  } catch (error) {
    res.status(500).json({ message: 'Server error', error: error.message
  });
  }
};

module.exports = { register, login };

```

5. Frontend API Service (api.js)

This file configures the Axios instance used by the React frontend to communicate with the backend. It automatically attaches the JWT token to every outgoing request via an interceptor.

```
import axios from 'axios';

const API = axios.create({
  baseURL: 'https://smart-project-backend-production.up.railway.app'
});

API.interceptors.request.use((req) => {
  const token = localStorage.getItem('token');
  if (token) req.headers.Authorization = `Bearer ${token}`;
  return req;
});

export const register = (data) => API.post('/api/auth/register', data);
export const login = (data) => API.post('/api/auth/login', data);
export const getMe = () => API.get('/api/users/me');
export const getProjects = () => API.get('/api/projects');
export const createProject = (data) => API.post('/api/projects', data);
export const getTasks = () => API.get('/api/tasks');
export const createTask = (data) => API.post('/api/tasks', data);
export const updateTask = (id, data) => API.put(`/api/tasks/${id}`, data);
export const getMyNotifications = () => API.get('/api/notifications');
export const getTeamReport = () => API.get('/api/reports/team');
```

6. Frontend Routing (App.js)

This file defines the React Router routes for the application. It includes a PrivateRoute component that redirects unauthenticated users to the login page, protecting the Leader and Member dashboards.

```
import React from 'react';
import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom';
import Home from './pages/Home';
import Login from './pages/Login';
import Register from './pages/Register';
import LeaderDashboard from './pages/LeaderDashboard';
import MemberDashboard from './pages/MemberDashboard';

const PrivateRoute = ({ children }) => {
  const token = localStorage.getItem('token');
  return token ? children : <Navigate to="/login" />;
};

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/login" element={<Login />} />
        <Route path="/register" element={<Register />} />
        <Route path="/leader" element={
          <PrivateRoute><LeaderDashboard /></PrivateRoute>
        } />
      </Routes>
    </BrowserRouter>
  );
}
```

```

    <Route path="/member"    element={
      <PrivateRoute><MemberDashboard /></PrivateRoute>} />
    </Routes>
  </BrowserRouter>
);
}

export default App;

```

Appendix B — User Satisfaction Survey Results

A user satisfaction survey was distributed to 71 participants to evaluate the Smart Project Management System. The survey covered system usability, feature usefulness, technical choices, and overall satisfaction. Below are the key findings.

1.Participant Profile (N = 71)

Table B.1: Participant Roles

Role	Count (%)
Team Member / Employee	23 (32.4%)
Student / Researcher	20 (28.2%)
Other	16 (22.5%)
Team Leader / Project Manager	10 (14.1%)
Academic Supervisor / Reviewer	2 (2.8%)

2.Quantitative Ratings (Scale: 1–5)

Table B.2: Quantitative Survey Results

Question	Average Rating
Importance of a centralized platform (tasks, time, performance)	4.00 / 5
Importance of the problem (lack of unified system)	3.77 / 5
Technology stack evaluation (React.js, Node.js, PostgreSQL, JWT)	3.75 / 5
Quality of project documentation (report + Swagger API docs)	3.85 / 5
Overall evaluation as an academic graduation project	4.20 / 5

3.System Adoption Intent

Table B.3: Would You Use This System?

Response	Count (%)
Yes, I would definitely use it	40 (56.3%)
Maybe, after adding some features	16 (22.5%)
No, I prefer existing tools	8 (11.3%)
Not applicable	7 (9.9%)

4. Deployment Readiness Assessment

Table B.4: Deployment Readiness

Assessment	Count (%)
Strong prototype, needs further development before deployment	28 (39.4%)
Ready for immediate production deployment	23 (32.4%)
Needs minor security and performance improvements	14 (19.7%)
Needs restructuring of some core components	6 (8.5%)

In summary, the survey results indicate strong overall acceptance of the system. The average overall rating was 4.20 out of 5, and 56.3% of participants confirmed they would use the system in a real team environment. The results validate the project's core value proposition and confirm that the system addresses a real need in team-based project management.